45. Host Network Performance Tuning Intro

45. Host Network Performance Tuning Intro

Phase: Phase 11 — Observability and troubleshooting
Estimated time: 1–2 days
Slug: `45-performance-tuning`

Learning objectives

- Inspect RX/TX errors and drops
- Know ephemerial port exhaustion exists
- Tune carefully with evidence
- Relate to ulimit/somaxconn

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

High connection churn exhausts sockets. Softirq, backlog, ephemeral ports, and buffer tuning — intro only, with 'measure first' discipline.

Core concepts

What you must be able to explain:

- Inspect RX/TX errors and drops
- Know ephemerial port exhaustion exists
- Tune carefully with evidence
- Relate to ulimit/somaxconn

Mental model for `45-performance-tuning` (exercise kind `perf_sysctl`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
ping -c 2 1.1.1.1; dig example.com +short; curl -sS -o /dev/null -w
'%{http_code}\n' https://example.com; ss -tln | head
```

Sample output:

```
icmp ok / A record / 200 / LISTEN lines
```

Interpretation: Playbook order: local stack → L3 → DNS → L4 listen/connect → L7. Capstones and interviews reward this sequence.

Example 2

```
ip route get 1.1.1.1; traceroute -n 1.1.1.1 2>/dev/null | head -4 || true
```

Sample output:

```
1.1.1.1 via 192.168.1.1 dev eth0 src 192.168.1.20
1 192.168.1.1
```

Interpretation: `ip route get` is the fastest 'which path would this host take?' check on Linux.

Example 3

```
curl -sS -o /dev/null -w 'namelookup=%{time_namelookup} connect=%{time_connect}
ttfb=%{time_starttransfer}\n' https://example.com
```

Sample output:

```
namelookup=0.020 connect=0.040 ttfb=0.120
```

Interpretation: Percentiles beat averages for latency SLOs; single curl samples teach which phase dominates.

Mini-lab

1. Re-type every worked command for **Host Network Performance Tuning Intro** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
sysctl cargo cult from blogs	See common mistakes	Re-run lesson commands
Raising limits hiding leaks	See common mistakes	Re-run lesson commands
Ignoring NIC hardware offload/errors	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for Host Network Performance Tuning Intro

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- sysctl cargo cult from blogs
- Raising limits hiding leaks
- Ignoring NIC hardware offload/errors

Exercises

Work in `exercises/45-performance-tuning/`. Read that folder's `README.md` before coding.

```
cd exercises/45-performance-tuning
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `measure_then`, `kernel_knobs_iface`, `blind_tuning_is`. Each function is a tiny retrieval drill for **Host Network Performance Tuning Intro** — return the exact tokens/flags/labels from the lesson. Keep the comment above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/45-performance-tuning/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. Where see interface drops?
2. Ephemeral ports why?
3. Measure before tune?

Further reading

- https://www.kernel.org/doc/Documentation/networking/ip-sysctl.txt